



[DOGFOOD 2026 · THE SPEC]

FOUR FILES. SEVEN CHECKS.

The website tells you what to build and how it is scored.
This page is the small list of things your project has to
agree on with ours, so a program can check your work.

Almost nothing here is a restriction. Most of it is a
picture.

[NEW TO SPECS? START HERE]

3,574 WORDS · FITS ONE MESSAGE

Copies this whole brief as plain text. Paste it into
ChatGPT or Claude and ask anything.

COPY EVERYTHING

>>>

> "Explain this like I have never entered a
hackathon"

> "What can I legally do before Friday?"

> "What should a team of two build first?"

> "Sketch me a database schema for T1"

> "What does role isolation actually mean?"

> "Write me a docker-compose for this"

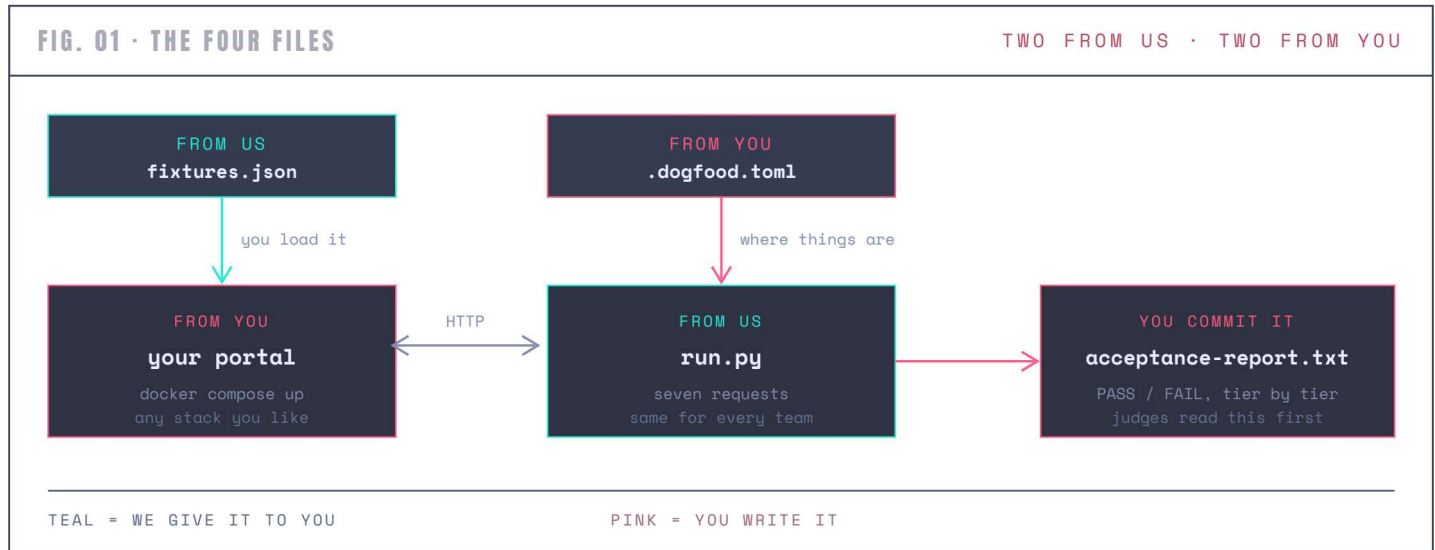
[DOWNLOAD SPEC.MD]

[DOWNLOAD RUN.PY]

[DOWNLOAD FIXTURES.JSON]

HOW IT ALL FITS TOGETHER

You build a portal. We give you fake data to put in it and a small program that pokes it. The program writes a report. You commit the report. That is the entire mechanism.



[WHAT YOU DO]

- + Load **fixtures.json** into your portal when it boots.
- + Write **.dogfood.toml**, about ten lines, at your repo root.
- + Run **run.py** and save what it prints.
- + Commit the report, whatever it says.

[WHAT WE DO NOT ASK]

- X No fixed API routes. Yours are yours.
- X No required framework, language or database.
- X No login format. We never log in.
- X No schema. Load the fixtures however you like.

02 [02 / YOUR REPO]

WHAT WE OPEN WHEN WE CLONE YOU

Three files at the root are ours. The documents are listed on the main site. Everything else is yours, arranged however you like.

FIG. 02 · REPO ROOT

[\[COPY \]](#)

```
your-repo/
├─ .dogfood.toml      ← where things are, and what you claim
├─ acceptance-report.txt ← what run.py printed
├─ docker-compose.yml ← one command to a working portal
├─ README.md          ← what it does, how to run, honest limits
├─ ARCHITECTURE.md     ← how it is put together, and why
├─ DATA-MODEL.md      ← your schema, and the way in and out
├─ JUDGING.md          ← assignment, scoring, normalization
├─ LICENSE             ← any OSI approved one
├─ src/                ← yours. any shape.
└─ tests/              ← yours. any shape.
```

Plus the five minute demo video, linked wherever you like.

03 [\[03 / .DOGFOOD.TOML \]](#)

TEN LINES THAT MAKE YOU CHECKABLE

Forty teams are building forty different portals. If we demanded a fixed set of routes we would be designing your API for you. So we do the opposite: build what you want, then write down where it ended up.

.DOGFOOD.TOML[\[COPY \]](#)

```
[portal]
base_url = "http://localhost:8080"

[tiers]
claimed = ["T1", "T2"]
pitch = "One sentence on what you built."

[auth]
# Whatever header proves you are this role. A cookie, a
# bearer token, a basic auth string. Your seed script
```

```
# prints these when the portal boots.
organizer    = "Cookie: session=org_7f2a"
judge_a      = "Cookie: session=jdg_a_91bc"
judge_b      = "Cookie: session=jdg_b_44de"
participant  = "Cookie: session=prt_2e88"

[routes]
gallery      = "/projects"
submit       = "/projects/new"
judge_scores = "/api/judge/scores"
peer_scores  = "/api/judge/scores?judge=judge_a"
csv_export   = "/api/export.csv"
```

[AUTH]

The checker never logs in. Logging in is the one thing no two stacks do alike, so instead you hand over a working header and we attach it. Your login page can work however you want. Make your seed script print these four headers when the portal boots and this takes a minute to fill in.

[PEER_SCORES]

This is the url that, in your portal, would return judge A's scores. A query parameter, a path like `/api/judges/jdg_01/scores`, anything. We visit it as judge B, who should be turned away. We ask you for the url rather than dictating one, because your API is yours.

claimed is on your honour, and it is checked. Put down the tiers you believe you finished. The checker prints what it verified next to what you claimed. Claiming T1 and verifying T1 is a clean result. Saying you got further than you did is the one thing that actually costs points.

04 [04 / FIXTURES.JSON]

THE SAME FAKE DATA IN EVERY PORTAL

If everyone seeds their own demo data, a judge opening two portals is comparing test data, not software. One team seeds three tidy projects, another seeds forty messy ones, and the tidy one looks better for no good reason. Shared fixtures remove that.

It contains awkward cases on purpose: a judge who gave every project the same score, two review batches nobody finished, one duplicate submission. Those happen at real hackathons and how your portal copes is interesting.

FIXTURES.JSON · SHAPE[\[COPY \]](#)

```
{
  "event": {
    "id": "evt_01",
    "name": "Sample Hack 2026",
    "submissions_close": "2026-03-01T18:00:00Z"
  },
  "tracks": [ { "id": "trk_01", "name": "Developer tools" } ],
  "judges": [ { "id": "jdg_01", "name": "Ada Okonkwo",
    "email": "ada@example.org", "tracks": ["trk_01"] } ],
  "teams": [ { "id": "tm_01", "name": "Nightshift",
    "members": ["ada@example.org"] } ],
  "projects": [ { "id": "prj_01", "team": "tm_01", "track": "trk_01",
    "title": "Quiet Hours", "summary": "One line.",
    "repo_url": "https://example.org/repo",
    "submitted_at": "2026-02-28T22:14:00Z" } ],
  "scores": [ { "judge": "jdg_01", "project": "prj_01",
    "criteria": { "functionality": 4, "quality": 3 },
    "comment": "Text, sometimes empty." } ]
}
```

Every id is a string. Every timestamp is ISO 8601 in UTC. Some score entries are missing on purpose. Your portal should not fall over when one project has two reviews and its neighbour has five.

You do not have to store it in this shape. Load it, transform it, put it in whatever schema you can defend. The file is input, not your data model.

[IT IS ALREADY HERE]

We are not holding anything back. The real file, all forty projects and every awkward case, is downloadable right now. Load it, poke at it, design your schema against the actual data rather than the sample.

[\[DOWNLOAD FIXTURES.JSON \]](#)

EVERYTHING RUN.PY ASKS YOUR PORTAL

All seven, in full. Nothing hidden, nothing extra. Each is behaviour you were building anyway.

RUN IT

[COPY]

python3 run.py .dogfood.toml > acceptance-report.txt

Any Python 3, including the one already on your Mac. Standard library only, nothing to install. Keep fixtures.json next to run.py and it will find it.

T1 · A stranger can browse the gallery. GET {routes.gallery} no auth header expect 200	T1 · The gallery shows fixture projects. GET {routes.gallery} expect a known fixture title in the body
T1 · A closed event refuses submissions. POST {routes.submit} as participant expect 4xx	T2 · A judge can read their own scores. GET {routes.judge_scores} as judge_a expect 200
T2 · A judge cannot read a peer's scores. GET {routes.peer_scores} as judge_b expect 401 or 403	T2 · A participant is not a judge. GET {routes.judge_scores} as participant expect 401 or 403
T2 · An organizer can export CSV. GET {routes.csv_export} as organizer expect 200 and a CSV body	That is the whole list. The checker does not look at your HTML, your framework, your database, your file layout or your commit history.

On the closed event check: the fixture event's `submissions_close` is a date in the past, so if you seeded honestly your portal is already closed and this post has to be refused. It does not manipulate a

clock and it does not inspect why you refused. Seed with the fixture's close date rather than one of your own and it passes first try.

THE ONE THAT COSTS THE MOST POINTS

Hiding another judge's scores in your template is not refusing. The check has to live in the backend, because the backend is where curl arrives.

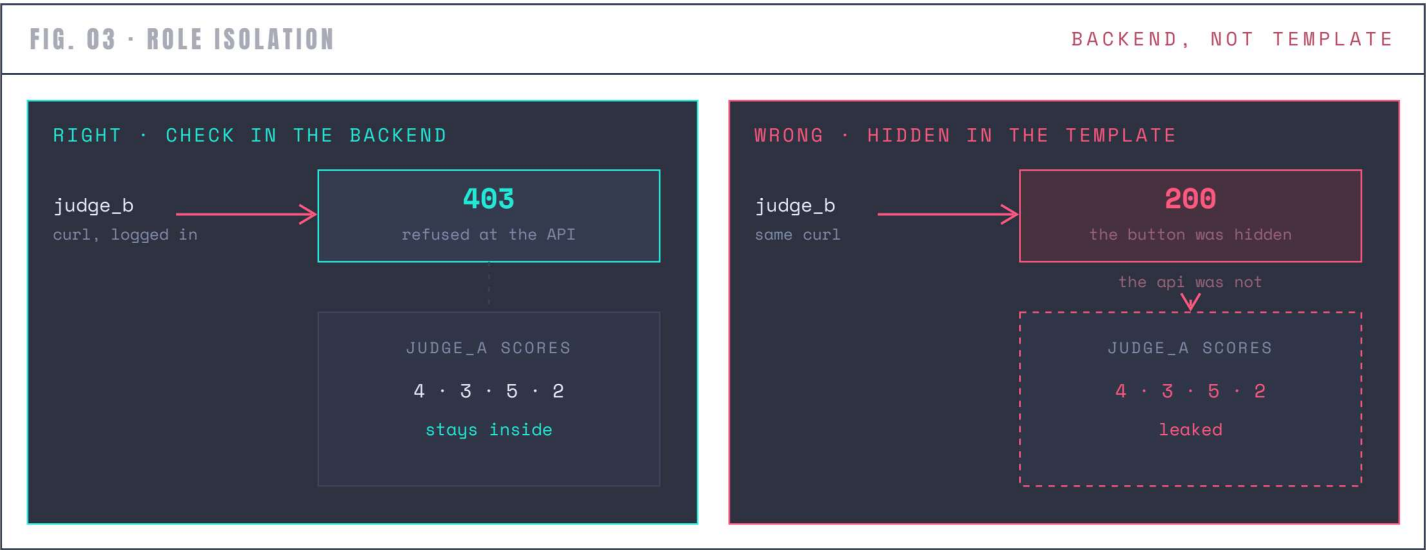


FIG. 04 · REPORT ANATOMY

[\[COPY \]](#)

```
DOGF00D 2026 acceptance report
portal: http://localhost:8080
claimed: T1 T2
fixtures: fixtures.json

T1 gallery is public ..... PASS
T1 project from fixtures shown ..... PASS
T1 closed event refuses submissions .. PASS
T2 judge sees own scores ..... PASS
T2 judge cannot see peer scores ..... FAIL      ← 2
    GET http://localhost:8080/api/judge/scores?judge=judge_a
    sent as judge_b; this is the url that returns judge_a's scores
    got 200, wanted 401 or 403
    the backend returned another judge's scores
T2 participant blocked ..... PASS
T2 csv export works ..... PASS      ← 1

claimed T1 T2, verified T1      ← 3
note: claimed but not verified: T2
```

1 · The verdict column

One word per check. This is what a judge scans.

2 · The detail under a FAIL

The exact request, who sent it, what came back and what was wanted. Enough to fix it without guessing.

3 · The last line

What you claimed against what was verified. A gap here is the only thing that costs you.

Commit the report even when it has failures in it. A report with two honest **FAIL** lines reads better than a README claiming everything works. Write the gaps into your README too, in your own words. We run the same checker, so there is no advantage in hiding anything.

07

[07 / THE LADDER]

CLIMB AS FAR AS YOU GET

The full version is on the main site. This is the pocket copy. T1 is the floor. A clean T2 beats a broken T4, because correctness is worth more than breadth.



08

[08 / WHEN]

SEVENTY TWO HOURS





[BEFORE KICKOFF]

- + Read this page and the main site.
- + Pick a stack you already know.
- + Sketch a schema against the fixtures shape above.
- + Form your team. One to four people, solo is fine.

[NOT BEFORE KICKOFF]

X Project code. That starts Friday at 18:00 UTC.

Every file is already on this page, `run.py` and `fixtures.json` included. Nothing drops on Friday. Download them now, read the checker, load the data, and arrive on Friday knowing exactly what you are building toward.

Planning, reading and sketching are all fine and encouraged. Frameworks, libraries, boilerplate generators and AI tools are fine too. New code only means you do not arrive with the portal already written.

09 [09 / BONUS POINTS]

THEY BREAK TIES. THEY DO NOT ADD UP.

Your score is the weighted average of the four published criteria, on a scale of **0 to 5**. **Bonus points do not change it.** What they do is separate two projects that land on the same number, and decide the Best Judging Engine prize.

There are four of them on the main site: a normalization proof, a pairwise judging mode, a threat model, and an API first design. Attempt any, all, or none. Do them because the problems are genuinely interesting, or because you want an edge in a photo finish. Not because you are chasing arithmetic.

10 [10 / YOUR CALL]

WHAT WE DO NOT CHECK

Worth saying out loud, because listed as rules it would be most of this page.

Your language. Your framework. Your database. Your ORM, or the absence of one. Your schema. Your route names. Your CSS, your component library, your lack of a component library. Your repo layout. Your commit style. Your branch names. Whether you wrote tests and how many. Whether you used Claude Code, Cursor, Copilot or none of them. How you split work between teammates. Whether you sleep.

Pick what you know. A boring stack you are fluent in will get further in 72 hours than an exciting one you are learning.

11 [11 / THE FIVE THINGS]

EVERYTHING THAT IS ACTUALLY REQUIRED

THE CHECKLIST

[\[COPY ALL \]](#)

- 1 `docker compose up` brings up a working, seeded portal with the network off. No cloud accounts, no hosted database, no external API.
- 2 An `OSI approved license` in the repo. MIT or Apache-2.0 preferred.
- 3 `Code written during the event window`. Frameworks, libraries, boilerplate generators and AI tools are all fine.
- 4 `.dogfood.toml` at the repo root, with honest tier claims.
- 5 `acceptance-report.txt` committed, whatever it says.

Plus the documents: `README.md`, `ARCHITECTURE.md`, `DATA-MODEL.md`, `JUDGING.md`, and the five minute demo video.

[\[DOWNLOADS \]](#)[\[SPEC.MD · THE FULL TEXT \]](#)[\[RUN.PY · THE CHECKER \]](#)[\[EXAMPLE .DOGFOOD.TOML \]](#)[\[CONTEXT.TXT · EVERYTHING IN ONE FILE \]](#)[\[FIXTURES.JSON · THE REAL DATA \]](#)[\[IF SOMETHING HERE IS UNCLEAR \]](#)

Ask in Discord. If a question comes up twice, this page gets a line added and everyone sees the same answer. An ambiguity found on Wednesday is a fixed spec. The same one found on Saturday is three teams building three different things.

[\[GO TO DISCORD \] >>>](#)

DOGFOOD®

A `raptors.dev` hackathon. Build the submission and judging platform we will actually run. Open source, self-hosted, yours.

[\[GO TO DISCORD \]](#)

THIS PAGE

FOUR FILES
THE CHECKS
TIERS
WHEN
REQUIRED

THE EVENT

ABOUT
TIERS
SCORING
PRIZES
RULES
FAQ

CONNECT

hello@raptors.dev

Hackathon Raptors

Community Interest Company 15557917

Office 2131, 182-184 High Street North,
East Ham, London, E6 2JA, United Kingdom

© 2026 DOGFOOD HACKATHON · BUILT BY RAPTORS.DEV

+ 51.5310°N 0.0500°E + REV 2.6 + UNIT / DF-01™